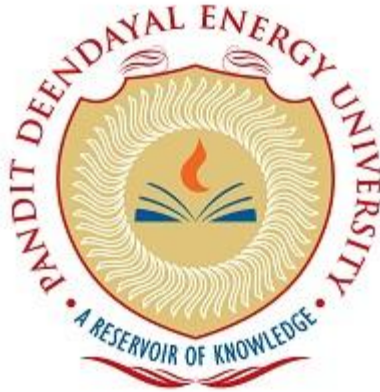




Pandit Deendayal Energy University (PDEU)
Department of Computer Science and Engineering,
School of Technology,
Design and Analysis of Algorithm
(24CS210P)

Name : Jay Patel

Roll No : 24BCP291

Lab Experiment 1

Objective:

Implementing the sorting algorithm: Bubble sort, Selection sort, Insertion sort, Merge sort and measuring the execution time and the number of steps of each.

Code:

```
#include <stdio.h>
#include <time.h>
long long steps = 0;
void merge(int a[], int l, int m, int r)
{
    int i, j, k;
    steps++;
    int n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++)
    {
        L[i] = a[l + i];
        steps++;
    }
    for (j = 0; j < n2; j++)
    {
        R[j] = a[m + 1 + j];
        steps++;
    }
    i = 0;
    j = 0;
    k = l;
    while (i < n1 && j < n2)
    {
        steps++;
        if (L[i] <= R[j])
            a[k++] = L[i++];
        else
            a[k++] = R[j++];
    }
    while (i < n1)
    {
```

```

        a[k++] = L[i++];
        steps++;
    }
    while (j < n2)
    {
        a[k++] = R[j++];
        steps++;
    }
}

```

```

void mergesort(int a[], int l, int r)
{
    steps++;
    if (l < r)
    {
        int m = (l + r) / 2;
        mergesort(a, l, m);
        mergesort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

```

```

int partition(int arr[], int low, int high)
{
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++)
    {
        steps++;
        if (arr[j] < pivot)
        {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
}

```

```

        steps++;
    }
}
int temp = arr[i + 1];
arr[i + 1] = arr[high];
arr[high] = temp;
return i + 1;
}

void quicksort(int arr[], int low, int high)
{
    steps++;
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quicksort(arr, low, pi - 1);
        quicksort(arr, pi + 1, high);
    }
}

void copy(int original[], int duplicate[], int n)
{
    for (int i = 0; i < n; i++)
        duplicate[i] = original[i];
}

int main()
{
    clock_t start, end;
    double time_taken;
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int original[n];

```

```

printf("Enter elements:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &original[i]);
int arr[n], ar[n], a[n], A[n], q[n];

copy(original, arr, n);
steps = 0;
start = clock();
for (int i = 0; i < n - 1; i++)
{
    int idx = i;
    for (int j = i + 1; j < n; j++)
    {
        steps++;
        if (arr[idx] > arr[j])
            idx = j;
    }
    if (idx != i)
    {
        int temp = arr[idx];
        arr[idx] = arr[i];
        arr[i] = temp;
        steps++;
    }
}
end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nSelection Sort: ");
for (int i = 0; i < n; i++)
    printf("%d ", arr[i]);
printf("\nSteps=%lld Time=%f sec\n", steps, time_taken);

copy(original, ar, n);
steps = 0;

```

```

start = clock();
for (int i = 0; i < n; i++)
    for (int j = 0; j < n - 1 - i; j++)
    {
        steps++;
        if (ar[j] > ar[j + 1])
        {
            int temp = ar[j];
            ar[j] = ar[j + 1];
            ar[j + 1] = temp;
            steps++;
        }
    }
end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nBubble Sort: ");
for (int i = 0; i < n; i++)
    printf("%d ", ar[i]);
printf("\nSteps=%lld Time=%f sec\n", steps, time_taken);

```

```

copy(original, a, n);
steps = 0;
start = clock();
for (int i = 1; i < n; i++)
{
    int key = a[i], j = i - 1;
    while (j >= 0 && a[j] > key)
    {
        a[j + 1] = a[j];
        j--;
        steps++;
    }
    a[j + 1] = key;
}

```

```
end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nInsertion Sort: ");
for (int i = 0; i < n; i++)
    printf("%d ", a[i]);
printf("\nSteps=%lld Time=%f sec\n", steps, time_taken);
```

```
copy(original, A, n);
steps = 0;
start = clock();
mergesort(A, 0, n - 1);
end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nMerge Sort: ");
for (int i = 0; i < n; i++)
    printf("%d ", A[i]);
printf("\nSteps=%lld Time=%f sec\n", steps, time_taken);
```

```
copy(original, q, n);
steps = 0;
start = clock();
quicksort(q, 0, n - 1);
end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nQuick Sort: ");
for (int i = 0; i < n; i++)
    printf("%d ", q[i]);
printf("\nSteps=%lld Time=%f sec\n", steps, time_taken);
```

```
}
```

OUTPUT:

Enter number of elements: 6

Enter elements:

78

44

56

92

21

89

Selection Sort: 21 44 56 78 89 92

Steps=18 Time=0.000000 sec

Bubble Sort: 21 44 56 78 89 92

Steps=22 Time=0.000000 sec

Insertion Sort: 21 44 56 78 89 92

Steps=7 Time=0.000000 sec

Merge Sort: 21 44 56 78 89 92

Steps=48 Time=0.000000 sec

Quick Sort: 21 44 56 78 89 92

Steps=27 Time=0.000000 sec